



Hashing

Deep Dive + Real-World Use Cases + Complexity

Java 8+

THIRAVUGAL



Session Structure

1. What is hashing?
2. Hash function fundamentals
3. HashMap internal working
4. Collision handling
5. Time & space complexity
6. Real-world systems
7. Interview-level problems



What is Hashing?

Hashing is a technique to:

- Map data $\rightarrow$ fixed index
- Enable fast lookup
- Reduce search complexity

Goal:

Convert $O(n)$ search $\rightarrow O(1)$ average lookup.



Hash Function

A hash function:

- Takes key
- Produces integer hash
- Maps to bucket index

```
index = hash(key) % capacity
```

Good hash function:

- Fast
- Uniform distribution
- Minimal collisions



HashMap in Java

```
java.util.HashMap<String, Integer> map = new java.util.HashMap<>();
```

Operations:

- put() → O(1) average
- get() → O(1) average
- containsKey() → O(1) average

Worst case: O(n)



How HashMap Works Internally

1. Key.hashCode()
2. Compress hash
3. Find bucket index
4. Handle collision

Java 8 improvement:

- Converts long collision chains to balanced tree
- Worst case improves to $O(\log n)$



Collision Handling

Two main strategies:

1. Chaining (Linked List / Tree)
2. Open Addressing (Probing)

Java uses:

Chaining + Tree after threshold



Time Complexity

Operation	Average	Worst
put()	$O(1)$	$O(n)$
get()	$O(1)$	$O(n)$
remove()	$O(1)$	$O(n)$

Space Complexity: $O(n)$



Real World Use Case — Caching

Example:

User session cache.

Lookup by session ID.

Time: $O(1)$

Used in:

- Authentication systems
- API gateways
- Microservices

Real World Use Case — Frequency Counting

Example:

Count word frequency in logs.

```
static java.util.Map<String, Integer> frequency(String[] words) {  
    java.util.Map<String, Integer> map = new java.util.HashMap<>();  
    for (String w : words) {  
        map.put(w, map.getOrDefault(w, 0) + 1);  
    }  
    return map;  
}
```

Time: $O(n)$

Two Sum Problem

Time: $O(n)$

Space: $O(n)$

```
static int[] twoSum(int[] nums, int target) {  
    java.util.Map<Integer, Integer> map = new java.util.HashMap<>();  
  
    for (int i = 0; i < nums.length; i++) {  
        int complement = target - nums[i];  
        if (map.containsKey(complement))  
            return new int[]{map.get(complement), i};  
        map.put(nums[i], i);  
    }  
    return new int[]{};  
}
```



Anagram Grouping

Time: $O(n * k \log k)$

Space: $O(n)$

```
static java.util.List<java.util.List<String>> groupAnagrams(String[] strs) {
    java.util.Map<String, java.util.List<String>> map = new java.util.HashMap<>();

    for (String s : strs) {
        char[] arr = s.toCharArray();
        java.util.Arrays.sort(arr);
        String key = new String(arr);

        map.computeIfAbsent(key, k -> new java.util.ArrayList<>()).add(s);
    }
    return new java.util.ArrayList<>(map.values());
}
```



Subarray Sum Equals K

Time: $O(n)$

Space: $O(n)$

```
static int subarraySum(int[] nums, int k) {  
    java.util.Map<Integer, Integer> map = new java.util.HashMap<>();  
    map.put(0, 1);  
  
    int sum = 0, count = 0;  
  
    for (int num : nums) {  
        sum += num;  
        if (map.containsKey(sum - k))  
            count += map.get(sum - k);  
  
        map.put(sum, map.getOrDefault(sum, 0) + 1);  
    }  
    return count;  
}
```



HashSet

Used for uniqueness.

```
java.util.HashSet<Integer> set = new java.util.HashSet<>();
```

Time: $O(1)$ average for `add()`, `contains()`

Use cases:

- Remove duplicates
- Cycle detection
- Membership testing



Common Mistakes

- Forgetting equals() & hashCode()
- Using mutable keys
- Ignoring load factor
- Assuming $O(1)$ always

equals() and hashCode()

Contract:

If two objects are equal $\rightarrow$ hashCode must be equal.

Example:

```
class Person {
    String name;
    int age;

    @Override
    public boolean equals(Object o) {
        Person p = (Person) o;
        return name.equals(p.name) && age == p.age;
    }

    @Override
    public int hashCode() {
        return name.hashCode() + age;
    }
}
```




Load Factor

Default load factor: 0.75

When:

$\text{size} > \text{capacity} \times \text{loadFactor}$

HashMap resizes.

Resizing cost: $O(n)$



Interview Discussion Points

- Why average $O(1)$?
- When does it degrade?
- Why Java 8 tree conversion?
- What happens during resize?
- Why immutable keys are recommended?



Practice Problems

Easy:

- Contains duplicate
- Intersection of arrays

Medium:

- Top K frequent elements
- Longest consecutive sequence

Advanced:

- LRU Cache
- Design Twitter
- Randomized Set



Summary

Hashing enables:

- $O(1)$ average lookup
- Efficient frequency counting
- Optimized subarray problems
- Real-world caching systems

Master:

- Two Sum
- Prefix sum + HashMap
- Collision handling
- equals & hashCode contract